



Search

1.5



DOCUMENTATION

BLOG

HISTORY



Commands

You've already learned how to use the command-line interface to do some things. This chapter documents all the available commands.

To get help from the command-line, simply call `poetry` to see the complete list of commands, then `--help` combined with any of those can give you more information.

Global options

```
--verbose (-v|-vv|-vvv) : Increase the verbosity of messages: "-v" for normal output, "-vv" for more verbose output and "-vvv" for debug.
--help (-h) : Display help information.
--quiet (-q) : Do not output any message.
--ansi : Force ANSI output.
--no-ansi : Disable ANSI output.
--version (-V) : Display this application version.
--no-interaction (-n) : Do not ask any interactive question.
--no-plugins : Disables plugins.
--no-cache : Disables Poetry source caches.
--directory=DIRECTORY (-C) : The working directory for the Poetry command (defaults to the current working directory).
```

new

This command will help you kickstart your new Python project by creating a directory structure suitable for most projects.

```
poetry new my-package
```

will create a folder as follows:

```
my-package
├── pyproject.toml
├── README.md
├── my_package
│   └── __init__.py
└── tests
    └── __init__.py
```

If you want to name your project differently than the folder, you can pass the `--name` option:

```
poetry new my-folder --name my-package
```

If you want to use a src folder, you can use the `--src` option:

```
poetry new --src my-package
```

That will create a folder structure as follows:

```
my-package
├── pyproject.toml
├── README.md
├── src
│   └── my_package
│       └── __init__.py
└── tests
    └── __init__.py
```

The `--name` option is smart enough to detect namespace packages and create the required structure for you.

```
poetry new --src --name my.package my-package
```

will create the following structure:

```
my-package
├── pyproject.toml
├── README.md
├── src
│   ├── my
│   └── package
│       └── __init__.py
└── tests
    └── __init__.py
```

Options

```
--name : Set the resulting package name.
--src : Use the src layout for the project.
--readme : Specify the readme file extension. Default is md. If you intend to publish to PyPI keep the recommendations for a PyPI-friendly README in mind.
```

init

This command will help you create a `pyproject.toml` file interactively by prompting you to provide basic information about your package.

It will interactively ask you to fill in the fields, while using some smart defaults.

```
poetry init
```

Options

```
--name : Name of the package.
--description : Description of the package.
--author : Author of the package.
--python : Compatible Python versions.
--dependency : Package to require with a version constraint. Should be in format foo:1.0.0.
--dev-dependency : Development requirements, see --dependency.
```

install

The `install` command reads the `pyproject.toml` file from the current project, resolves the dependencies, and installs them.

```
poetry install
```

If there is a `poetry.lock` file in the current directory, it will use the exact versions from there instead of resolving them. This ensures that everyone using the library will get the same versions of the dependencies.

If there is no `poetry.lock` file, Poetry will create one after dependency resolution.

If you want to exclude one or more dependency groups for the installation, you can use the `--without` option.

```
poetry install --without test,docs
```

✓

The `--no-dev` option is now deprecated. You should use the `--without dev` notation instead.

You can also select optional dependency groups with the `--with` option.

```
poetry install --with test,docs
```

It's also possible to only install specific dependency groups by using the `only` option.

```
poetry install --only test,docs
```

To only install the project itself with no dependencies, use the `--only-root` flag.

```
poetry install --only-root
```

See [Dependency groups](#) for more information about dependency groups.

If you want to synchronize your environment – and ensure it matches the lock file – use the `--sync` option.

```
poetry install --sync
```

The `--sync` can be combined with group-related options:

```
poetry install --without dev --sync
poetry install --with docs --sync
poetry install --only dev
```

You can also specify the extras you want installed by passing the `-E|--extras` option (See [Extras](#) for more info). Pass `--all-extras` to install all defined extras for a project.

```
poetry install --extras "mysql psql"
poetry install -E mysql -E psql
poetry install --all-extras
```

Extras are not sensitive to `--sync`. Any extras not specified will always be removed.

```
poetry install --extras "A B" # C is removed
```

By default, `poetry` will install your project's package every time you run `install`:

```
$ poetry install
Installing dependencies from lock file

No dependencies to install or update

- Installing <your-package-name> (x.x.x)
```

If you want to skip this installation, use the `--no-root` option.

```
poetry install --no-root
```

Similar to `--no-root` you can use `--no-directory` to skip directory path dependencies:

```
poetry install --no-directory
```

This is mainly useful for caching in CI or when building Docker images. See the [FAQ](#) entry for more information on this option.

By default, `poetry` does not compile Python source files to bytecode during installation. This speeds up the installation process, but the first execution may take a little more time because Python then compiles source files to bytecode automatically. If you want to compile source files to bytecode during installation, you can use the `--compile` option:

```
poetry install --compile
```

✓

The `--compile` option has no effect if `installer.modern-installation` is set to `false` because the old installer always compiles source files to bytecode.

Options

- `--without`: The dependency groups to ignore.
- `--with`: The optional dependency groups to include.
- `--only`: The only dependency groups to include.
- `--only-root`: Install only the root project, exclude all dependencies.
- `--sync`: Synchronize the environment with the locked packages and the specified groups.
- `--no-root`: Do not install the root package (your project).
- `--no-directory`: Skip all directory path dependencies (including transitive ones).
- `--dry-run`: Output the operations but do not execute anything (implicitly enables `--verbose`).
- `--extras (-E)`: Features to install (multiple values allowed).
- `--all-extras`: Install all extra features (conflicts with `--extras`).
- `--compile`: Compile Python source files to bytecode.
- `--no-dev`: Do not install dev dependencies. (**Deprecated**, use `--without dev` or `--only main` instead)
- `--remove-untracked`: Remove dependencies not presented in the lock file. (**Deprecated**, use `--sync` instead)

✓

When `--only` is specified, `--with` and `--without` options are ignored.

update

In order to get the latest versions of the dependencies and to update the `poetry.lock` file, you should use the `update` command.

```
poetry update
```

This will resolve all dependencies of the project and write the exact versions into `poetry.lock`.

If you just want to update a few packages and not all, you can list them as such:

```
poetry update requests toml
```

Note that this will not update versions for dependencies outside their version constraints specified in the `pyproject.toml` file. In other terms, `poetry update foo` will be a no-op if the version constraint specified for `foo` is `~2.3` or `2.3` and 2.4 is available. In order for `foo` to be updated, you must update the constraint, for example `>2.3`. You can do this using the `add` command.

Options

- `--without`: The dependency groups to ignore.
- `--with`: The optional dependency groups to include.
- `--only`: The only dependency groups to include.
- `--dry-run`: Outputs the operations but will not execute anything (implicitly enables `--verbose`).
- `--no-dev`: Do not update the development dependencies. (**Deprecated**, use `--without dev` or `--only main` instead)
- `--lock`: Do not perform install (only update the lockfile).

✓

When `--only` is specified, `--with` and `--without` options are ignored.

add

Commands | Documentation | Poetry - Python dependency management and packaging made easy

The `add` command adds required packages to your `pyproject.toml`, and installs them.

If you do not specify a version constraint, poetry will choose a suitable one based on the available package versions.

```
poetry add requests pendulum
```

You can also specify a constraint when adding a package:

```
# Allow >=2.0.5, <3.0.0 versions
poetry add pendulum<^2.0.5

# Allow >=2.0.5, <2.1.0 versions
poetry add pendulum~2.0.5

# Allow >=2.0.5 versions, without upper bound
poetry add "pendulum>=2.0.5"

# Allow only 2.0.5 version
poetry add pendulum==2.0.5
```

See the [Dependency specification](#) page for more information about the `@` operator.

If you try to add a package that is already present, you will get an error. However, if you specify a constraint, like above, the dependency will be updated by using the specified constraint.

If you want to get the latest version of an already present dependency, you can use the special `latest` constraint:

```
poetry add pendulum@latest
```

See the [Dependency specification](#) for more information on setting the version constraints for a package.

You can also add `git` dependencies:

```
poetry add git+https://github.com/sdispater/pendulum.git
```

or use ssh instead of https:

```
poetry add git+ssh://git@github.com/sdispater/pendulum.git

# or alternatively:
poetry add git+ssh://git@github.com:sdispater/pendulum.git
```

If you need to checkout a specific branch, tag or revision, you can specify it when using `add`:

```
poetry add git+https://github.com/sdispater/pendulum.git#develop
poetry add git+https://github.com/sdispater/pendulum.git#2.0.5

# or using SSH instead:
poetry add git+ssh://git@github.com/sdispater/pendulum.git#develop
poetry add git+ssh://git@github.com/sdispater/pendulum.git#2.0.5
```

or reference a subdirectory:

```
poetry add git+https://github.com/myorg/mypackage_with_subdirs.git#main#subdirectory
```

You can also add a local directory or file:

```
poetry add ./my-package/
poetry add ../my-package/dist/my-package-0.1.0.tar.gz
poetry add ../my-package/dist/my_package-0.1.0.whl
```

If you want the dependency to be installed in editable mode you can use the `--editable` option.

```
poetry add --editable ./my-package/
poetry add --editable git+ssh://github.com/sdispater/pendulum.git#develop
```

Alternatively, you can specify it in the `pyproject.toml` file. It means that changes in the local directory will be reflected directly in environment.

```
[tool.poetry.dependencies]
my-package = {path = "../my/path", develop = true}
```

Before poetry 1.1 path dependencies were installed in editable mode by default. You should always set the `develop` attribute explicitly, to make sure the behavior is the same for all poetry versions.

If the package(s) you want to install provide extras, you can specify them when adding the package:

```
poetry add "requests[security,socks]"
poetry add "requests[security,socks]>=2.22.0"
poetry add "git+https://github.com/pallets/flask.git@1.1.1[dotenv,dev]"
```

Some shells may treat square braces (`[` and `]`) as special characters. It is suggested to always quote arguments containing these characters to prevent unexpected shell expansion.

If you want to add a package to a specific group of dependencies, you can use the `--group (-G)` option:

```
poetry add mkdocs --group docs
```

See [Dependency groups](#) for more information about dependency groups.

Options

- `--group (-G)`: The group to add the dependency to.
- `--dev (-D)`: Add package as development dependency. (**Deprecated**, use `-G dev` instead)
- `--editable (-e)`: Add vcs/path dependencies as editable.
- `--extras (-E)`: Extras to activate for the dependency. (multiple values allowed)
- `--optional`: Add as an optional dependency.
- `--python`: Python version for which the dependency must be installed.
- `--platform`: Platforms for which the dependency must be installed.
- `--source`: Name of the source to use to install the package.
- `--allow-prereleases`: Accept prereleases.
- `--dry-run`: Output the operations but do not execute anything (implicitly enables `--verbose`).
- `--lock`: Do not perform install (only update the lockfile).

remove

The `remove` command removes a package from the current list of installed packages.

```
poetry remove pendulum
```

If you want to remove a package from a specific group of dependencies, you can use the `--group (-G)` option:

```
poetry remove mkdocs --group docs
```

See [Dependency groups](#) for more information about dependency groups.

Options

- `--group (-G)` : The group to remove the dependency from.
- `--dev (-D)` : Removes a package from the development dependencies. (**Deprecated**, use `-G dev` instead)
- `--dry-run` : Outputs the operations but will not execute anything (implicitly enables `--verbose`).
- `--lock` : Do not perform operations (only update the lockfile).

show

To list all the available packages, you can use the `show` command.

```
poetry show
```

If you want to see the details of a certain package, you can pass the package name.

```
poetry show pendulum

name      : pendulum
version   : 1.4.2
description : Python datetimes made easy

dependencies
- python-dateutil >=2.6.1
- tzlocal >=1.4
- pytzdata >=2017.2.2

required by
- calendar >=1.4.0
```

Options

- `--without` : The dependency groups to ignore.
- `--why` : When showing the full list, or a `--tree` for a single package, display why a package is included.
- `--with` : The optional dependency groups to include.
- `--only` : The only dependency groups to include.
- `--no-dev` : Do not list the dev dependencies. (**Deprecated**, use `--without dev` or `--only main` instead)
- `--tree` : List the dependencies as a tree.
- `--latest (-l)` : Show the latest version.
- `--outdated (-o)` : Show the latest version but only for packages that are outdated.
- `--all (-a)` : Show all packages (even those not compatible with current system).
- `--top-level (-T)` : Only show explicitly defined packages.



When `--only` is specified, `--with` and `--without` options are ignored.

Introduction

Basic usage

Managing dependencies

Libraries

Commands

Global options

new

Options

init

Options

install

Options

update

Options

add

Options

remove

Options

show

Options

build

Options

publish

Options

config

Usage

Options

run

shell

check

search

lock

Options

version

Options

export

Options

env

cache

cache list

cache clear

source

source add

source show

source remove

about

help

list

self

self add

self update

self lock

self show

self show plugins

self remove

self install

Configuration

Repositories

Managing environments

Dependency specification

Plugins

The pyproject.toml file

Contributing to Poetry

FAQ

pre-commit hooks



build

The `build` command builds the source and wheels archives.

```
poetry build
```

Note that, at the moment, only pure python wheels are supported.

Options

- `--format (-F)` : Limit the format to either `wheel` or `sdist`.

publish

This command publishes the package, previously built with the `build @` command, to the remote repository.

It will automatically register the package before uploading if this is the first time it is submitted.

```
poetry publish
```

It can also build the package if you pass it the `--build` option.



See Publishable Repositories for more information on how to configure and use publishable repositories.

Options

- `--repository (-r)` : The repository to register the package to (default: `pypi`). Should match a repository name set by the `config @` command.
- `--username (-u)` : The username to access the repository.
- `--password (-p)` : The password to access the repository.
- `--cert` : Certificate authority to access the repository.
- `--client-cert` : Client certificate to access the repository.
- `--build` : Build the package before publishing.
- `--dry-run` : Perform all actions except upload the package.
- `--skip-existing` : Ignore errors from files already existing in the repository.

config

The `config` command allows you to edit poetry config settings and repositories.

```
poetry config --list
```

Usage

```
poetry config [options] [setting-key] [setting-value1] ... [setting-valueN]
```

`setting-key` is a configuration option name and `setting-value1` is a configuration value. See Configuration for all available settings.

Options

- `--unset` : Remove the configuration element named by `setting-key`.
- `--list` : Show the list of current config variables.
- `--local` : Set/Get settings that are specific to a project (in the local configuration file `poetry.toml`).

run

The `run` command executes the given command inside the project's virtualenv.

```
poetry run python -V
```

It can also execute one of the scripts defined in `pyproject.toml`.

So, if you have a script defined like this:

```
[tool.poetry.scripts]
my-script = "my_module:main"
```

You can execute it like so:

```
poetry run my-script
```

Note that this command has no option.

shell

The `shell` command spawns a shell, according to the `$SHELL` environment variable, within the virtual environment. If one doesn't exist yet, it will be created.

```
poetry shell
```

Note that this command starts a new shell and activates the virtual environment.

As such, `exit` should be used to properly exit the shell and the virtual environment instead of `deactivate`.

check

The `check` command validates the structure of the `pyproject.toml` file and returns a detailed report if there are any errors.



This command is also available as a pre-commit hook. See pre-commit hooks for more information.

```
poetry check
```

search

This command searches for packages on a remote index.

```
poetry search requests pendulum
```

lock

This command locks (without installing) the dependencies specified in `pyproject.toml`.



By default, this will lock all dependencies to the latest available compatible versions. To only refresh the lock file, use the `--no-update` option. This command is also available as a pre-commit hook. See pre-commit hooks for more information.

```
poetry lock
```

Options

```
--check : Verify that poetry.lock is consistent with pyproject.toml
--no-update : Do not update locked versions, only refresh lock file.
```

version

This command shows the current version of the project or bumps the version of the project and writes the new version back to `pyproject.toml` if a valid bump rule is provided.

The new version should be a valid PEP 440 string or a valid bump rule: `patch`, `minor`, `major`, `prepatch`, `preminor`, `premajor`, `prerelease`.



If you would like to use semantic versioning for your project, please see [here](#).

The table below illustrates the effect of these rules with concrete examples.

RULE	BEFORE	AFTER
major	1.3.0	2.0.0
minor	2.1.4	2.2.0
patch	4.1.1	4.1.2
premajor	1.0.2	2.0.0a0
preminor	1.0.2	1.1.0a0
prepatch	1.0.2	1.0.3a0
prerelease	1.0.2	1.0.3a0
prerelease	1.0.3a0	1.0.3a1
prerelease	1.0.3b0	1.0.3b1

Options

```
--short (-s) : Output the version number only.
--dry-run : Do not update pyproject.toml file.
```

export

This command exports the lock file to other formats.

```
poetry export -f requirements.txt --output requirements.txt
```



This command is provided by the Export Poetry Plugin and is also available as a pre-commit hook. See pre-commit hooks for more information.



Unlike the `install` command, this command only includes the project's dependencies defined in the implicit `main` group defined in `tool.poetry.dependencies` when used without specifying any options.

Options

```
--format (-f) : The format to export to (default: requirements.txt). Currently, only constraints.txt and requirements.txt are supported.
--output (-o) : The name of the output file. If omitted, print to standard output.
--dev : Include development dependencies. (Deprecated, use --with dev instead)
--extras (-E) : Extra sets of dependencies to include.
--without : The dependency groups to ignore.
--with : The optional dependency groups to include.
--only : The only dependency groups to include.
--without-hashes : Exclude hashes from the exported file.
```

- `--without-urls` : Exclude source repository uris from the exported file.
- `--with-credentials` : Include credentials for extra indices.

env

The `env` command regroups sub commands to interact with the virtualenvs associated with a specific project.

See Managing environments for more information about these commands.

cache

The `cache` command regroups sub commands to interact with Poetry's cache.

cache list

The `cache list` command lists Poetry's available caches.

```
poetry cache list
```

cache clear

The `cache clear` command removes packages from a cached repository.

For example, to clear the whole cache of packages from the `pypi` repository, run:

```
poetry cache clear pypi --all
```

To only remove a specific package from a cache, you have to specify the cache entry in the following form `cache:package:version`:

```
poetry cache clear pypi:requests:2.24.0
```

source

The `source` namespace regroups sub commands to manage repository sources for a Poetry project.

source add

The `source add` command adds source configuration to the project.

For example, to add the `pypi-test` source, you can run:

```
poetry source add pypi-test https://test.pypi.org/simple/
```

You cannot use the name `pypi` for a custom repository as it is reserved for use by the default PyPI source. However, you can set the priority of PyPI:

```
poetry source add --priority=explicit pypi
```

Options

- `--default` : Set this source as the default (disable PyPI). Deprecated in favor of `--priority`.
- `--secondary` : Set this source as a secondary source. Deprecated in favor of `--priority`.
- `--priority` : Set the priority of this source. Accepted values are: `default` `secondary` `supplemental` `explicit`. Refer to the dedicated sections in Repositories for more information.

At most one of the options above can be provided. See package sources for more information.

source show

The `source show` command displays information on all configured sources for the project.

```
poetry source show
```

Optionally, you can show information of one or more sources by specifying their names.

```
poetry source show pypi-test
```

This command will only show sources configured via the `pyproject.toml` and does not include the implicit default PyPI.

source remove

The `source remove` command removes a configured source from your `pyproject.toml`.

```
poetry source remove pypi-test
```

about

The `about` command displays global information about Poetry, including the current version and version of `poetry-core`.

```
poetry about
```

help

The `help` command displays global help, or help for a specific command.

To display global help:

```
poetry help
```

To display help for a specific command, for instance `show`:

```
poetry help show
```

The `--help` option can also be passed to any command to get help for a specific command.

For instance:

```
poetry show --help
```

list

```
poetry list
```

self

The `self` namespace regroups sub commands to manage the Poetry installation itself.



Use of these commands will create the required `pyproject.toml` and `poetry.lock` files in your configuration directory.

self add

The `self add` command installs Poetry plugins and make them available at runtime. Additionally, it can also be used to upgrade Poetry's own dependencies or inject additional packages into the runtime environment



The `self add` command works exactly like the `add` `@` command. However, is different in that the packages managed are for Poetry's runtime environment.

The package specification formats supported by the `self add` command are the same as the ones supported by the `add` `@` command.

For example, to install the `poetry-plugin-export` plugin, you can run:

```
poetry self add poetry-plugin-export
```

To update to the latest `poetry-core` version, you can run:

```
poetry self add poetry-core@latest
```

To add a keyring provider `artifacts-keyring`, you can run:

```
poetry self add artifacts-keyring
```

Options

- `--editable (-e)` : Add vcs/path dependencies as editable.
- `--extras (-E)` : Extras to activate for the dependency. (multiple values allowed)
- `--allow-prereleases` : Accept prereleases.
- `--source` : Name of the source to use to install the package.
- `--dry-run` : Output the operations but do not execute anything (implicitly enables `--verbose`).

self update

The `self update` command updates Poetry version in its current runtime environment.



The `self update` command works exactly like the `update` `@` command. However, is different in that the packages managed are for Poetry's runtime environment.

```
poetry self update
```

Options

- `--preview` : Allow the installation of pre-release versions.
- `--dry-run` : Output the operations but do not execute anything (implicitly enables `--verbose`).

self lock

The `self lock` command reads this Poetry installation's system `pyproject.toml` file. The system dependencies are locked in the corresponding `poetry.lock` file.

```
poetry self lock
```

Options

- `--check` : Verify that `poetry.lock` is consistent with `pyproject.toml`.
- `--no-update` : Do not update locked versions, only refresh lock file.

self show

The `self show` command behaves similar to the `show` command, but working within Poetry's runtime environment. This lists all packages installed within the Poetry install environment.

To show only additional packages that have been added via self add and their dependencies use `self show --addons`.

```
poetry self show
```

Options

- `--addons` : List only add-on packages installed.
- `--tree` : List the dependencies as a tree.
- `--latest (-L)` : Show the latest version.
- `--outdated (-o)` : Show the latest version but only for packages that are outdated.

self show plugins

The `self show plugins` command lists all the currently installed plugins.

```
poetry self show plugins
```

self remove

The `self remove` command removes an installed addon package.

```
poetry self remove poetry-plugin-export
```

Options

- `--dry-run` : Outputs the operations but will not execute anything (implicitly enables `--verbose`).

self install

The `self install` command ensures all additional packages specified are installed in the current runtime environment.



The `self install` command works similar to the `install` `@` command. However, is different in that the packages managed are for Poetry's runtime environment.

```
poetry self install --sync
```



Options

- sync : Synchronize the environment with the locked packages and the specified groups.
- dry-run : Output the operations but do not execute anything (implicitly enables --verbose).



Python packaging and dependency management made easy.



DOCUMENTATION

- Introduction
- Basic usage
- Managing dependencies
- Libraries
- Commands
- Configuration
- Repositories
- Managing environments
- Dependency specification
- Plugins
- The pyproject.toml file
- Contributing to Poetry
- FAQ
- pre-commit hooks

GITHUB

- Project
- Issues
- Discussions

OTHER PROJECTS

- poetry-core
- install.python-poetry.org
- Bundle plugin
- Export plugin